

BR ESW Relay Controller Solution

Solution approach

The code is split into small C modules, similar to objects in OOP.

- **RelayController** — application logic only. It owns an array of relay instances and runs the global state machine (Init, On, Error). **Init** runs once in `RelayController_Init()`; each periodic `Process()` call supervises relays in **On**, or keeps them open in **Error** after a fault. On any relay fault it enters **Error**, forces all relays open once on entry, blocks further CLOSE requests every cycle, and stays there until re-init. Hardware access is not done here; it goes through `RelayIo_*`.
- **RelayInstance** — reusable single-relay control and supervision component. The controller holds one instance per relay. Each instance drives one DPO, samples one DI, compares feedback against the applied command every cycle, and latches a fault when a mismatch persists longer than the feedback settle window (30 ms).
- Struct members start with `_` (for example `_state`, `_fault`). Other code should only use the public `RelayController_*` and `RelayInstance_*` functions, not those fields directly.
- **HAL layer**: `relay_io.h` defines the DPO/DI interface (`RelayIo_*`). The build links either `relay_io_sim.c` (PC demo and tests) or `relay_io_hw.c` (target MCU), keeping application logic separate from the IO layer.
- **Status API**: `RelayController_GetContactState()`, `GetFault()`, `GetState()`, and `GetRequest()` expose relay feedback, faults, controller state, and stored requests. `RelayController_SetFaultNotifier()` is included as an example of optional push notification — it notifies the application once per fault when the controller enters Error. The same callback pattern could be extended to report per-relay contact or other state changes if needed.
- **Unit tests**: run `make test` — Google Test checks faults, bounce handling, scheduler, and the controller state machine.
- **Host demo**: CSV files in `simulation_scenarios/` show WELDED and CONSTANTLY_OPEN faults without real hardware.

Up to 10 relays are supported — one DPO and one DI per relay.

Every 5 ms, `Scheduler_RunTask()` runs `RelayController_Process()` once and steps through all relays. On the MCU that call would be inside a periodic RTOS task; on the PC demo, `main()` calls it in a loop and treats each call as one tick.

Two faults are detected when output and feedback disagree for 30 ms (six cycles):

- **WELDED** — commanded OPEN, feedback still closed
- **CONSTANTLY_OPEN** — commanded CLOSE, feedback still open

When any fault is latched, the controller enters **Error**: all relays are forced OPEN, CLOSE is blocked, and only `RelayController_Init()` clears the fault state.

The design uses three layers — application, relay control, and HAL.

NO / NC DPO mapping

Relay type	CLOSE (energized)	OPEN (de-energized)
NO	DPO HIGH	DPO LOW
NC	DPO LOW	DPO HIGH

Command	Expected DI feedback
CLOSE	DI HIGH (closed)
OPEN	DI LOW (open)

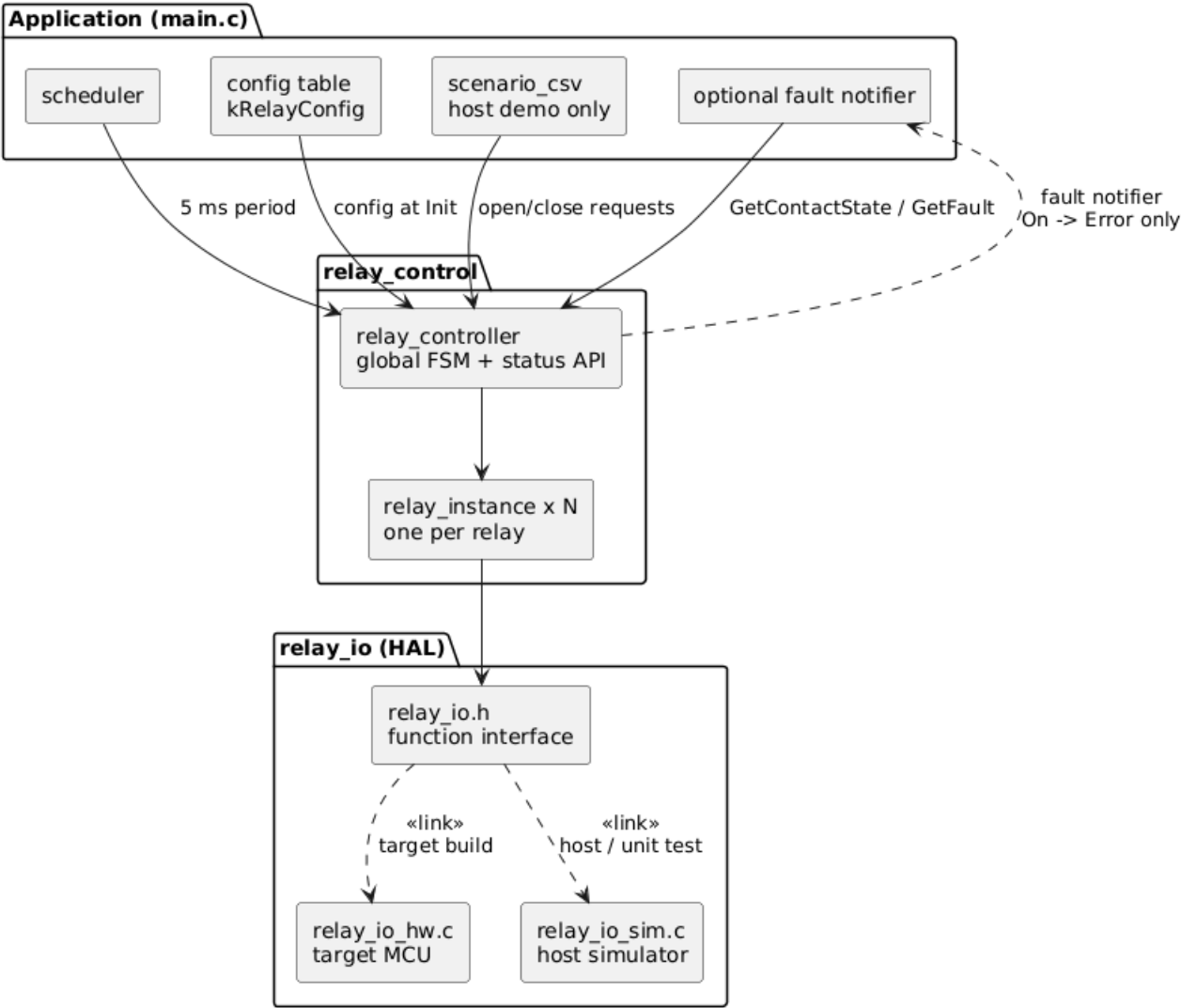
Assumptions

- Startup drives all relays OPEN

Module decomposition

Layer	Module	Responsibility
Application	main.c , scheduler , scenario_csv	Config, init, schedule, demo
Controller	relay_controller	Global FSM, error reaction, status API, optional fault notifier
Component	relay_instance	Per-relay command and supervision
HAL	relay_io	DPO/DI access (hw or sim, one per build)

Module decomposition

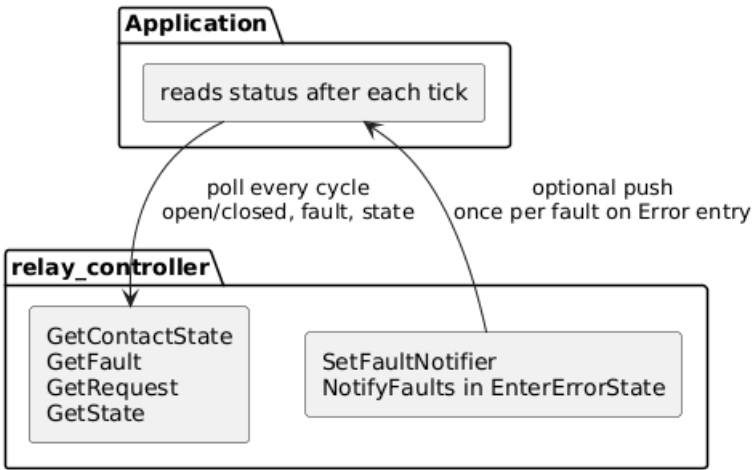


Status reporting

Poll (every cycle) — `GetContactState()` , `GetFault()` , `GetRequest()` , `GetState()` .

Optional fault notify (example) — `SetFaultNotifier()` is implemented as an example of push notification: called once per faulted relay from `EnterErrorState()` . The same pattern could be added for per-relay contact or state changes.

Status reporting — getters and optional fault notifier

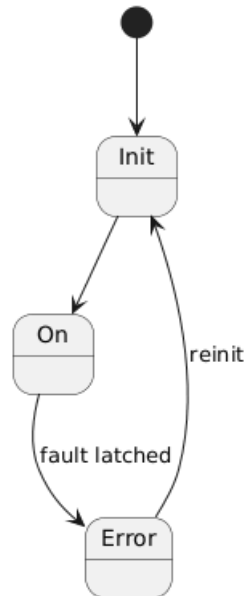


Controller state diagram

States: **Init**, **On**, **Error**

- **Init** — `RelayController_Init()` drives all relays OPEN, clears faults
- **On** — apply OPEN/CLOSE requests, supervise feedback
- **Error** — all relays forced OPEN, CLOSE blocked until re-init

Controller state diagram

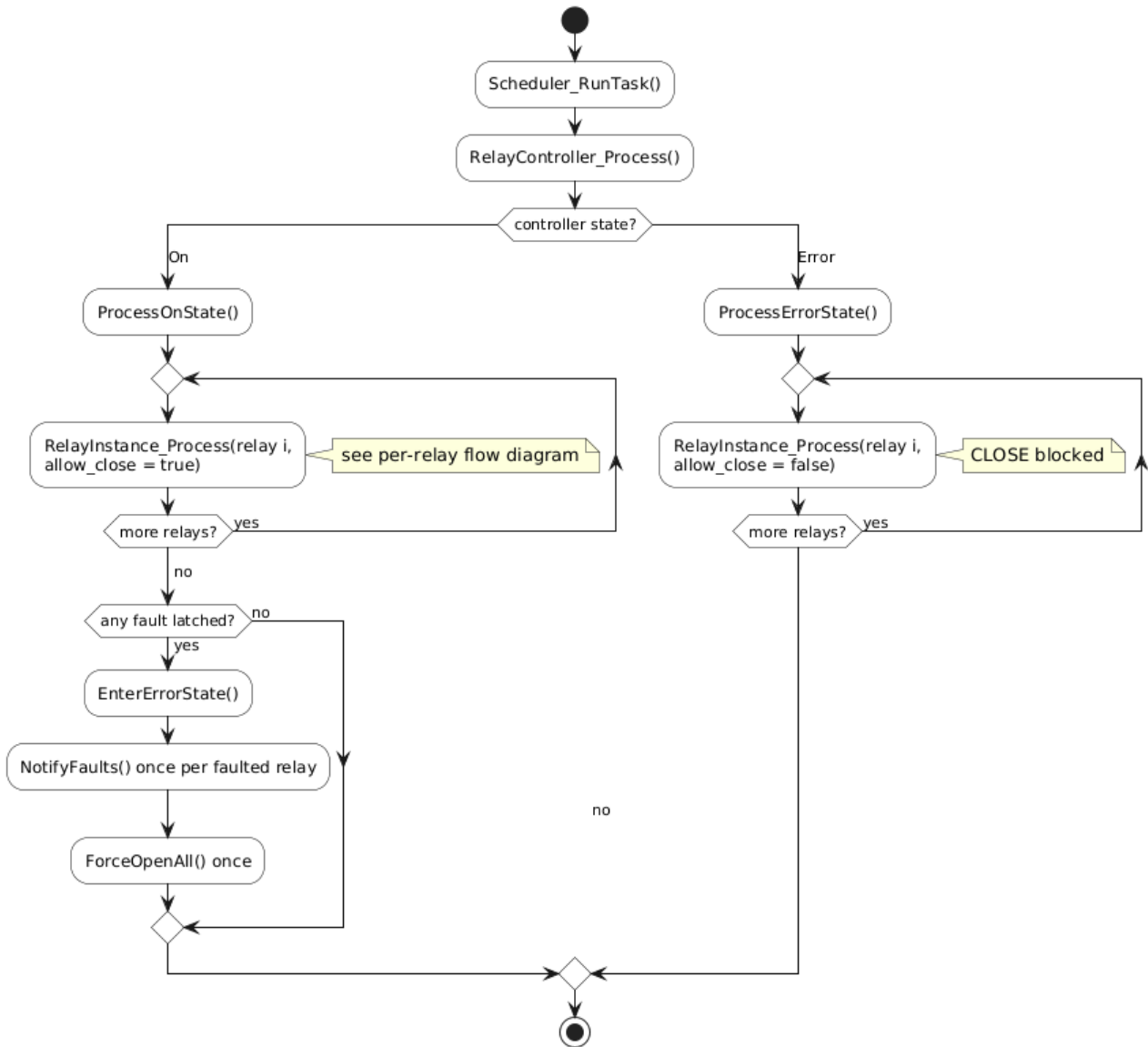


Controller execution flow

One 5 ms cycle at controller level:

1. Scheduler_RunTask() → RelayController_Process()
2. switch(state) : **On** → ProcessOnState() ; **Error** → ProcessErrorState()
3. Each handler runs RelayInstance_Process() for every relay (allow_close true in On, false in Error)
4. In On only: if any fault latched → EnterErrorState() → NotifyFaults() → ForceOpenAll() once

Controller execution flow (one 5 ms cycle)

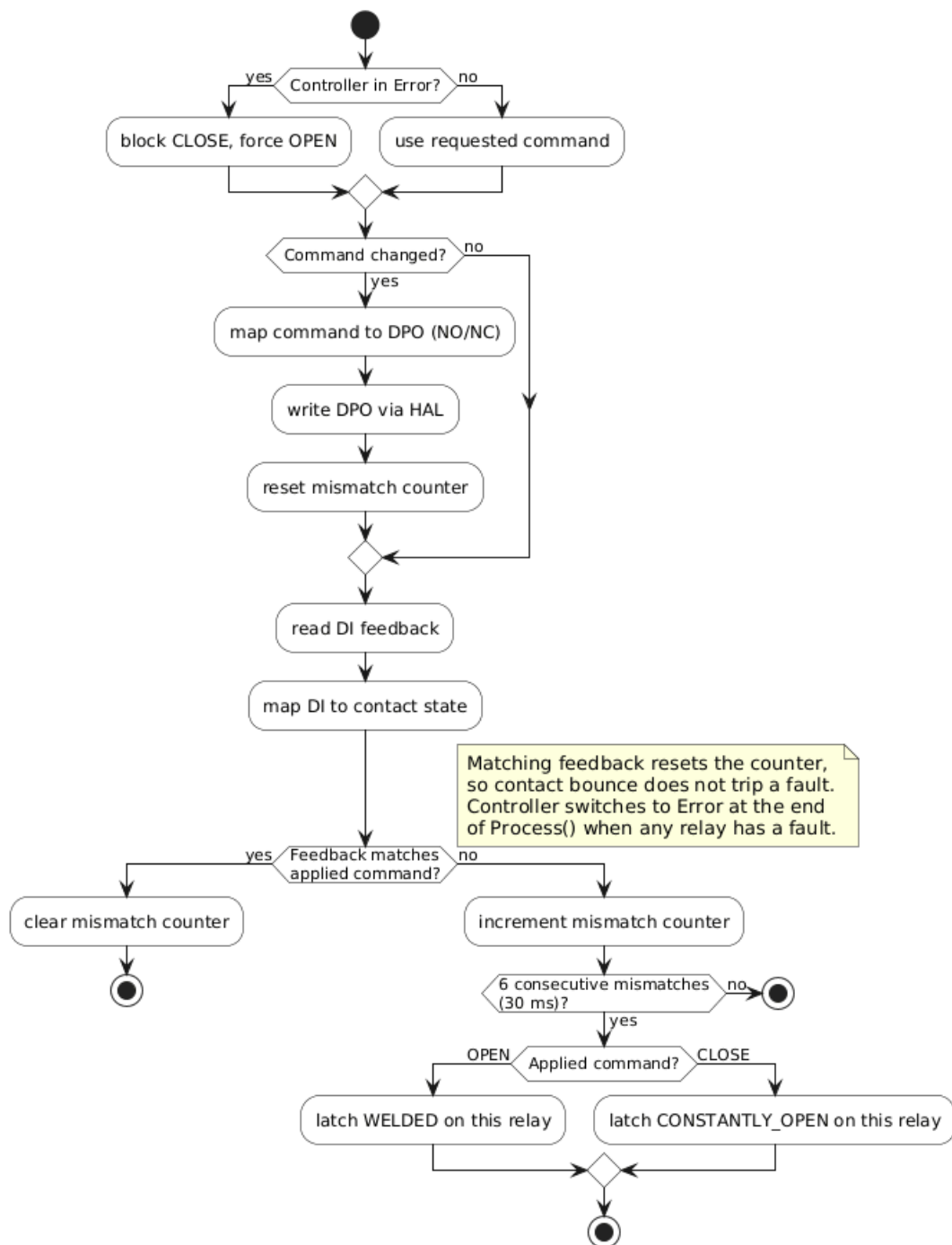


Per-relay execution flow

One 5 ms cycle for a single relay inside `RelayController_Process()` :

1. If controller in Error: block CLOSE, force OPEN
2. If command changed: map to DPO (NO/NC), write HAL, reset mismatch counter
3. Read DI feedback, map to contact state
4. If feedback matches applied command: clear mismatch counter
5. Else increment counter; after 6 consecutive mismatches latch fault
 - OPEN + closed feedback → **WELDED**
 - CLOSE + open feedback → **CONSTANTLY_OPEN**
6. Controller enters Error at end of `Process()` if any fault latched

Per-relay flow (one 5 ms cycle)



Scheduling and timing

- Task period: **5 ms** (`kRelayTaskPeriodMs`)
- DPO settle: **1 ms** (simulation)
- Fault window: **30 ms** (6 consecutive mismatch cycles)

Build, demo, and tests

```
make prod      # production build (no logging)
make debug     # host demo with LOG_* output
make test      # Google Test unit tests
```

Demo scenarios in `simulation_scenarios/` :

- `welded.csv` — WELDED fault on relay 0 (NO)
- `constantly_open.csv` — CONSTANTLY_OPEN on relay 1 (NC)

Sample debug demo output (make debug , also in `docs/simulation_log.txt`):

```
[INF] Relay controller initialized with 2 relay(s)
[WRN] Relay fault detected -> entering ERROR state, opening all relays
[WRN]   Relay_NO_1: WELDED
[INF] Relay controller initialized with 2 relay(s)
[WRN] Relay fault detected -> entering ERROR state, opening all relays
[WRN]   Relay_NC_1: CONSTANTLY_OPEN
```

Full C source, headers, config, and README are in the project repository.